

# P-384 scalar-inversion addition chain

Tommy Tai | Research-review candidate | 21 September 2026

**425 operations = 380 squarings + 45 other multiplications.**

Independent review pending. Not a production implementation or an accepted record.

## Claim and comparison

The sequence constructs the P-384 **scalar group order minus two**, not the coordinate-field prime minus two. Exponent 1 is the sole free starting value; all precomputation is counted. Each row uses previously available operands and has output = left + right. No operation is unused.

Compared with the identified **ring** scalar-inversion chain at commit 840167e18e4f: **2 fewer squarings and 3 fewer other multiplications** (430 to 425 operations). This is an operation-count comparison, not a measured runtime speedup.

## What we checked

Python exact integers, JavaScript BigInt, and C++ multiprecision agree on the exact endpoint and the 425 / 380 / 45 counts. Each rejects 20 negative controls (60 total), including corrupted dependencies, malformed input, and valid chains for the wrong exponent. A fixed panel of **1,413 nonzero scalars** passes modular inversion checks. OpenSSL independently supplies the scalar-order cross-check.

## Reproduce in minutes

The web directory contains the canonical and numerically sorted chains, three checker sources, a compact regeneration recipe, a Haskell expression, test results, and checksums. Start with the plain-text chain or run:

```
python verification/verify_python.py p384_scalar_inversion_425.txt
```

Run all three checkers and the full negative/regression panel with `python verification/check_review_package.py`. Requirements: Python 3.10+, Node.js with BigInt, a C++17 compiler, and Boost headers. OpenSSL is optional. No AI models, accounts, or original optimizer are needed.

## Method, next experiment, and limits

AI-assisted search explored precomputation dictionaries and overlapping positive-digit tail decompositions. The recipe regenerates the result; it does not recreate every search run. Further search can change intermediate values and dictionary/tail choices against the same frozen evaluator.

No global-optimality proof, exhaustive novelty claim, production security review, measured native speedup, or external acceptance is claimed. Haskell was not compiled in this packaging pass. The checker implementations and their assumptions still merit independent review.

## Canonical certificate SHA-256

14da9fd3bfa7e3e615c78c5d6cf73dc0262ea3ff65eb29553be502de37f0df05